

Cohort module two, two hours

Isolate Agent Work with an Execution Harness

The build session. By the end an agent can change code in their repository while their own branch stays exactly as they left it.

0:00 to 0:15

Reading each other's policies

Goal	The homework gets read by someone other than its author, which is where the vague lines get found.
Say	• Pair them and give a single instruction: find one line you could not act on without asking the author a question. • Bring two examples back to the whole group and rewrite them live. • Make the standard explicit. A policy is finished when a stranger could apply it the same way you would.
Run	Nothing. Documents on screen, in pairs.
Room does	Edits at least one line based on somebody else's confusion.
Ask	<i>Which line did your partner read differently from how you meant it?</i>

0:15 to 1:00

Building it together

Goal Everyone has a working create and remove cycle in their own project, written by them.

- Say**
- Write it in front of them from an empty file rather than opening the finished version. The cohort types along.
 - Stop at each decision and say what the alternative was: a clone instead of a worktree, a random name instead of a derived one, a global cleanup instead of a per-task one.
 - Have everyone prove the same property on their own repository before moving on, because a harness nobody has proven is a harness nobody trusts.
-

Run

```
$ node steps/02-execution/demo.ts create 7
$ git -C .worktrees/fq-7 status
$ git status
$ node steps/02-execution/demo.ts remove 7
```

- Room does**
- Creates a workspace, writes into it, and confirms the main checkout is untouched.
 - Removes it and confirms nothing is left behind.
-

Ask *How long did the first one take on your project, and what dominated that time?*

1:00 to 1:35

Reach, declared per task

Goal Each person has limits their own reviewer could read, and has watched one stop a run.

- Say**
- Insist on the placement before the content. Limits go where a reviewer will see them, not inside instructions to the agent.
 - Work through a task that legitimately needs more reach, so the rule is not simply deny everything.
 - Show the enforcement path and be honest about where it is advisory and where it is real.
-

Run

```
$ node steps/02-execution/demo.ts limits
$ node steps/02-execution/demo.ts limits --violate
```

- Room does**
- Watches a refused write name the path and the rule behind it.
 - Writes limits for one real task in their own project.
-

Ask *Which of your tasks genuinely needs the network, and how would you keep that from becoming the default?*

1:35 to 2:00

This week's build

Goal Everyone knows exactly what to produce and what will be checked on Monday.

- Say**
- State the deliverable as something observable: a task that runs start to finish in isolation on their repository.
 - Predict the two failures they will hit, so neither costs an evening: dependencies not installed in the new workspace, and generated files ignored by the clone.
-

Run

```
$ node steps/02-execution/demo.ts selftest
```

Room does Runs the self-check against their own harness and reads what it reports.

Ask *Post your isolation time in the channel when you have it running. We compare on Monday.*

1:35 to 1:50

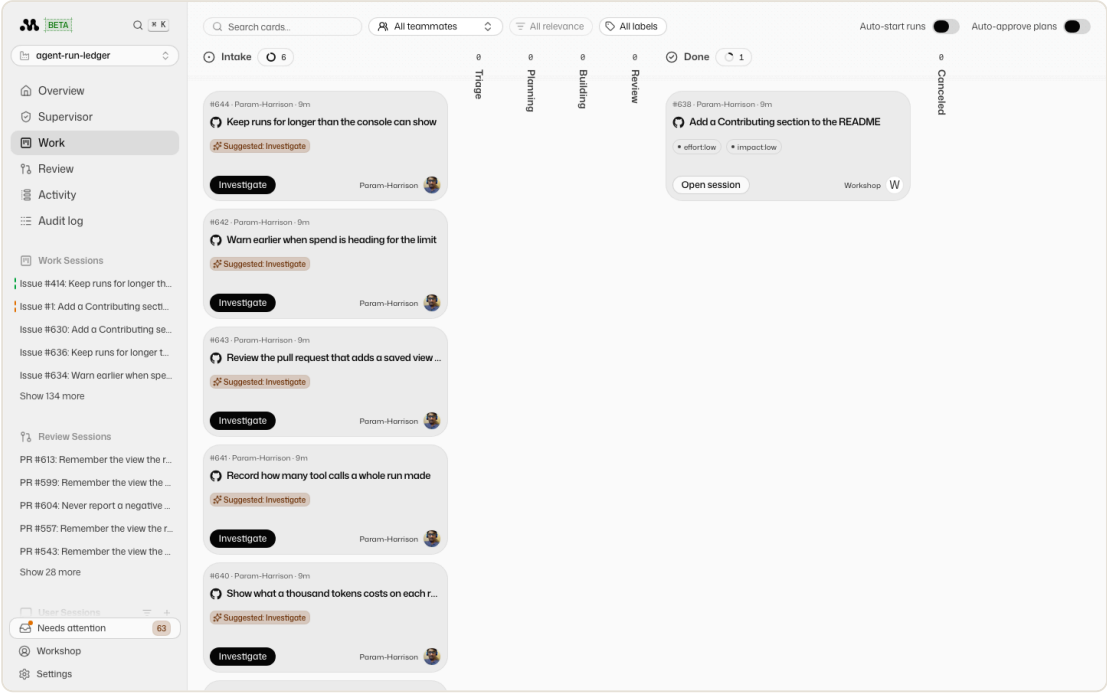
Many runs, one board

Goal They can explain why a queue limit is about attention rather than machine capacity.

- Say**
- Generation is cheap and review is not, so the constraint worth declaring is how many things may be waiting on a person.
 - Show the cap in the charter beside the board and let them find the mismatch if there is one.
 - A factory producing faster than a team can read is not more productive, it is further behind.

Run

```
$ cat $REPO/.factory/charter.md
```



The screenshot shows a web application titled 'agent-run-ledger' with a sidebar menu containing 'Overview', 'Supervisor', 'Work', 'Review', 'Activity', 'Audit log', 'Work Sessions', 'Needs attention' (63), 'Workshop', and 'Settings'. The main content area displays a list of work items, each with a title, a status (e.g., 'Suggested Investigate'), and an 'Investigate' button. The items are organized into columns representing different phases: Intake, Triage, Planning, Building, Review, and Cancelled. The 'Intake' column shows items like '#644: Param-Harrison - 9m' with the task 'Keep runs for longer than the console can show'. The 'Review' column shows items like '#639: Param-Harrison - 9m' with the task 'Add a Contributing section to the README'. The 'Cancelled' column shows items like '#638: Param-Harrison - 9m' with the task 'Add a Contributing section to the README'. The interface also includes search bars, filters, and a 'Done' button.

Several items in flight at once, each in a different phase.

Show on the board

Work to the board header to the phase counts to Settings

Work items come from events, never from polling

The reconcile sweep patches and closes items that exist; its counts have no created. The board fills because a webhook arrived, which is why a lab GitHub cannot reach delivers those events to itself.

The cap is about reading, not generating

Compare the number waiting with the stop condition in the charter. Generation is cheap; the constraint worth declaring is how many things may be queued against one person's attention.

Two boards, not one

Issues live on Work and pull requests live on Review, with their own phases. Teams that conflate them report a week of finished work that nobody has read.

Room does

- Finds the stop condition in the charter.
 - Counts what is waiting on the board and compares the two numbers.
-

Ask

At what number of open pull requests does your team stop reading them properly?

Concepts: the worktree, claiming by branch, declared limits, the executor.